МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені Тараса Шевченка
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
Кафедра програмних систем і технологій

Дисципліна
« Конструювання програмного забезпечення »

Лабораторна робота № 3

Виконав:	Берещук Назар Олександрович	**Перевірив**:	Шабатін П. Є.
Група	ІПЗ-34-10	Дата перевірки	
Форма навчання	денна	Оцінка	
Спеціальність	121		

2026

Варіант 1: Симулятор системи залежностей (Dependency Resolver)

Архітектура проекту (власні класи-обробники помилок):

Клас PackageManagerException для всіх власних виключень у проєкті.

Клас ParserException для всіх помилок, пов'язаних із розбором вхідного файлу.

Клас ValidationException для логічних помилок, які виникають після етапу парсингу.

Клас MissingDependencyException для ситуацій, коли один пакет вимагає інший пакет, але такого пакета немає у вхідному файлі.

Клас InvalidFormatException використовується у випадку, коли рядок у вхідному файлі має неправильний формат.

Клас InvalidVersionException відповідає за помилки формату версії.

Клас FileSystemException призначений для обробки помилок читання файлу.

Детально по класам:

PackageManagerException

Він успадковується від стандартного класу Exception і використовується для уніфікованої обробки помилок у програмі. Даний клас зберігає текст помилки та номер рядка, в якому вона виникла. Завдяки цьому користувач отримує зрозуміле повідомлення про помилку із зазначенням місця її появи.

```
public class PackageManagerException extends Exception  11 usages  Назар Берещук
{
    private int lineNumber;  2 usages

    public PackageManagerException(String message, int lineNumber)  3 usages  Назар Берещук
    {
        super(message);
        this.lineNumber = lineNumber;
    }

    public int getLineNumber() { return lineNumber; }
}
```

ParserException

Це проміжний базовий клас для всіх помилок, пов’язаних із розбором вхідного файлу. Він успадковується від PackageManagerException. Його призначення полягає у відокремленні синтаксичних помилок від інших типів помилок.

```
public class ParserException extends PackageManagerException  2 usages  2 inheritors  Назар Берещук
{
    public ParserException(String message, int lineNumber)  2 usages  Назар Берещук
    {
        super(message, lineNumber);
    }
}
```

ValidationException

Це базовий клас для логічних помилок, які виникають після етапу парсингу. Він успадковується від PackageManagerException і використовується для відокремлення семантичних помилок від синтаксичних.

```
public class ValidationException extends PackageManagerException  1 usage  1 inheritor  Назар Берещук
{
    public ValidationException(String message, int lineNumber)  1 usage  Назар Берещук
    {
        super(message, lineNumber);
    }
}
```

MissingDependencyException

Даний клас використовується для ситуацій, коли один пакет вимагає інший пакет, але такого пакета немає у вхідному файлі. Це логічна помилка, тому клас успадковується від ValidationException. Завдяки ньому програма може повідомити користувача, що залежність відсутня, замість аварійного завершення.

```
public class MissingDependencyException extends ValidationException  4 usages  Назар Берещук
{
    public MissingDependencyException(String message, int lineNumber)  1 usage  Назар Берещук
    {
        super(message, lineNumber);
    }
}
```

InvalidFormatException

Цей клас використовується у випадку, коли рядок у вхідному файлі має неправильний формат. Наприклад, якщо не вистачає версії пакета, відсутня кома, рядок має невідомий формат або конструкція requires: розташована раніше за package:. Клас успадковується від ParserException, адже це синтаксична помилка.

```java
public class InvalidFormatException extends ParserException  9 usages  Назар Берещук
{
    public InvalidFormatException(String message, int lineNumber)  4 usages  Назар Берещук
    {
        super(message, lineNumber);
    }
}
```

InvalidVersionException

Цей клас відповідає за помилки формату версії. Він використовується тоді, коли версія пакета не відповідає заданому шаблону v<number>.<number>. Наприклад, записи v1.x, 1.0 або v1 вважаються некоректними. Клас успадковується від ParserException, адже це синтаксична помилка.

```java
public class InvalidVersionException extends ParserException  4 usages  Назар Берещук
{
    public InvalidVersionException(String message, int lineNumber)  1 usage  Назар Берещук
    {
        super(message, lineNumber);
    }
}
```

FileSystemException

Цей клас відповідає за зчитування рядків із вхідного файлу. Якщо файл не вдається прочитати, стандартна помилка IOException перехоплюється та замінюється на власне виключення FileSystemException. Таким чином програма не завершується аварійно, а видає зрозуміле повідомлення користувачу. Успадковує базовий клас PackageManagerException.

```java
public class FileSystemException extends PackageManagerException  2 usages  Назар Берещук
{
    public FileSystemException(String message)  1 usage  Назар Берещук
    {
        super(message, -1);
    }
}
```

Архітектура проекту (класи для тестування):

Клас InputParserTest перевіряє роботу класу InputParser.

Клас ValidationEngineTest перевіряє логіку пошуку циклів у графі залежностей.

Клас ResolverEngineTest тестує алгоритм побудови порядку встановлення пакетів.

Клас InputReaderStubTest для тестування не справжнього файлу, шляхом використанні stub, підміни файлу.

Детально по класам:

InputParserTest

Позитивний тест:

1. перевірка, що парсер коректно обробляє правильний вхідний файл і будує граф залежностей.

```
@Test  Назар Берещук
void shouldParseValidInput() throws PackageManagerException
{
    InputParser parser = new InputParser();

    List<String> lines = List.of(
            "package: app, v1.0",
            "requires: core, v1.0",
            "package: core, v1.0"
    );

    DependencyGraph graph = parser.parse(lines);

    assertNotNull(graph);
    assertEquals( expected: 2, graph.getGraph().size());
}
```

Негативні тести:

1. перевірка виникнення InvalidVersionException при неправильному форматі версії;

```
@Test  Назар Берещук
void shouldThrowInvalidVersionExceptionForWrongVersion()
{
    InputParser parser = new InputParser();

    List<String> lines = List.of(
            e1: "package: app, v1.x"
    );

    InvalidVersionException exception = assertThrows(
            InvalidVersionException.class,
            () -> parser.parse(lines)
    );

    assertEquals( expected: 1, exception.getLineNumber());
}
```

2. перевірка виникнення InvalidFormatException при неправильному форматі рядка package:;

```
@Test  Назар Берещук
void shouldThrowInvalidFormatExceptionForWrongPackageFormat()
{
    InputParser parser = new InputParser();

    List<String> lines = List.of(
            e1: "package: app"
    );

    InvalidFormatException exception = assertThrows(
            InvalidFormatException.class,
            () -> parser.parse(lines)
    );

    assertEquals( expected: 1, exception.getLineNumber());
}
```

3. перевірка виникнення InvalidFormatException, якщо requires: зустрічається раніше за package:;

```
@Test  Назар Берещук
void shouldThrowInvalidFormatExceptionWhenRequiresComesFirst()
{
    InputParser parser = new InputParser();

    List<String> lines = List.of(
            "requires: core, v1.0",
            "package: app, v1.0"
    );

    InvalidFormatException exception = assertThrows(
            InvalidFormatException.class,
            () -> parser.parse(lines)
    );

    assertEquals( expected: 1, exception.getLineNumber());
}
```

4. перевірка виникнення MissingDependencyException, якщо залежність не оголошена у файлі.

```
@Test  Назар Берещук
void shouldThrowMissingDependencyExceptionForUnknownDependency()
{
    InputParser parser = new InputParser();

    List<String> lines = List.of(
            "package: app, v1.0",
            "requires: core, v1.0"
    );

    MissingDependencyException exception = assertThrows(
            MissingDependencyException.class,
            () -> parser.parse(lines)
    );

    assertEquals( expected: 2, exception.getLineNumber());
}
```

ValidationEngineTest

Цей клас перевіряє логіку пошуку циклів у графі залежностей.
У ньому реалізовано два основні сценарії:

1. перевірка, що у правильному графі циклів немає;
2. перевірка, що при циклічній залежності метод hasCycle() повертає true.

```
public class ValidationEngineTest  Назар Берещук
{
    @Test  Назар Берещук
    void shouldDetectNoCycle() throws PackageManagerException
    {
        InputParser parser = new InputParser();

        List<String> lines = List.of(
                "package: app, v1.0",
                "requires: core, v1.0",
                "package: core, v1.0"
        );

        DependencyGraph graph = parser.parse(lines);

        ValidationEngine validator = new ValidationEngine();

        assertFalse(validator.hasCycle(graph));
    }
```

```java
@Test  Назар Берещук
void shouldDetectCycle() throws PackageManagerException
{
    InputParser parser = new InputParser();

    List<String> lines = List.of(
            "package: app, v1.0",
            "requires: core, v1.0",
            "package: core, v1.0",
            "requires: app, v1.0"
    );

    DependencyGraph graph = parser.parse(lines);

    ValidationEngine validator = new ValidationEngine();

    assertTrue(validator.hasCycle(graph));
}
```

ResolverEngineTest

Цей клас тестує алгоритм побудови порядку встановлення пакетів.
Основне призначення тесту — перевірити, що залежності встановлюються раніше за пакети, які від них залежать. Наприклад, якщо пакет app залежить від core, то в порядку встановлення спочатку має бути core, а потім app.

```java
public class ResolverEngineTest  Назар Берещук
{
    @Test  Назар Берещук
    void shouldReturnCorrectInstallationOrder() throws PackageManagerException
    {
        InputParser parser = new InputParser();

        List<String> lines = List.of(
                "package: app, v1.0",
                "requires: core, v1.0",
                "package: core, v1.0"
        );

        DependencyGraph graph = parser.parse(lines);

        ResolverEngine resolver = new ResolverEngine();
        List<PackageId> order = resolver.topologicalSort(graph);

        assertEquals( expected: 2, order.size());
        assertEquals( expected: "core", order.get(0).getName());
        assertEquals( expected: "app", order.get(1).getName());
    }
}
```

InputReaderStubTest

Цей тестовий клас демонструє використання stub-об’єкта для ізоляції тестованого компонента від файлової системи.
У тесті створюється підклас StubInputReader, який не читає реальний файл, а повертає наперед визначений список рядків. Завдяки цьому перевіряється робота парсера без залежності від зовнішніх ресурсів.

```java
public class InputReaderStubTest  Назар Берещук
{
    @Test  Назар Берещук
    void shouldUseStubReaderInsteadOfRealFile() throws PackageManagerException
    {
        InputReader reader = new StubInputReader();
        InputParser parser = new InputParser();

        List<String> lines = reader.readFile( path: "ignored.txt");
        DependencyGraph graph = parser.parse(lines);

        assertEquals( expected: 2, graph.getGraph().size());
    }

    private static class StubInputReader extends InputReader  1 usage  Назар Берещук
    {
        @Override  2 usages  Назар Берещук
        public List<String> readFile(String path)
        {
            return List.of(
                    "package: app, v1.0",
                    "requires: core, v1.0",
                    "package: core, v1.0"
            );
        }
    }
}
```

Висновок:

Було реалізовано механізми захисного програмування шляхом створення ієрархії власних виключень та обробки помилок без аварійного завершення програми. Було забезпечено коректне виявлення синтаксичних і семантичних помилок у вхідних даних із виведенням зрозумілих повідомлень.

Також було розроблено модульні тести для перевірки як правильних, так і помилкових сценаріїв роботи програми. Для ізоляції компонентів використано stub-об’єкти. Налаштовано запуск тестів через Maven та забезпечено контроль якості коду.

Отримані результати підтверджують коректність роботи програми та її стійкість до некоректних вхідних даних.